From 360 .nat Files to a Paired PyTorch Dataset

Introduction

Having 360 downloaded `.nat` files is a solid foundation for training your semantic segmentation network.

Right now, you have two unaligned assets:

- **The Raw Inputs (X):** 360 heavy `.nat` files in `D:\FINDFIRE_DATA` (roughly 36 GB of multi-spectral satellite imagery).
- **The Target Labels (Y):** Thousands of GPS coordinates and timestamps stored in `nasa data set.xlsx`.

Here is how to match them automatically, project the coordinates into binary masks, split them chronologically to prevent data leakage, and package everything into an optimized PyTorch dataset.

1. The Core Engineering Goal

Your neural network cannot process a 100 MB `.nat` file or an Excel spreadsheet during training loops. It needs ready-to-load numerical arrays stored on your disk:

$$\text{Sample } i = \left(X_i \in \mathbb{R}^{3\times 120\times 250},\ Y_i \in \{0.0, 1.0\}^{120\times 250} \right)$$

- **Input (X_i):** 3 channels ($T_{3.9}$, $T_{10.8}$, and ΔT) cropped to Northern Algeria.
- **Target (Y_i):** 1-channel binary mask ($0.0 = \text{No Fire}, 1.0 = \text{Active Fire}$).

The Data Compression Advantage

A raw `.nat` file covers Europe and Northern Africa across 12 channels (1392×3712 pixels). When we isolate the 3 relevant channels and crop them to the Northern Algeria grid (120×250):

$$\text{Size per sample} = 4 \text{ matrices} \times 120 \times 250 \times 4 \text{ bytes (float32)} \approx 480 \text{ KB}$$

Compressed into a NumPy `.npz` archive, each sample is roughly 100 KB.

All 360 samples combined will take under 50 MB of storage! Once processed, the entire dataset will load into memory in seconds.

2. The Matching Algorithm (Temporal Pairing)

To match each `.nat` file on your drive to its corresponding rows in the Excel sheet without manual intervention, the script follows a 4-step algorithm:

```
1. Inspect .nat Filename          2. Query NASA DataFrame

MSG4-...-20260721121418...nat     All rows where:
Extracted: 2026-07-21 12:14       |T_nasa - T_sat| <= 15 min


Satellite Tensor X                Ground Truth Mask Y
  [3, 120, 250]                     [120, 250]


                Save 'sample_000.npz'
```

1. **Parse Satellite Datetime (T_{sat}):** EUMETSAT embeds the scan timestamp in the filename: `MSG4-SEVI-MSG15-0100-NA-YYYYMMDDHHMMSS...nat`. A regular expression extracts the year, month, day, hour, and minute.

2. **Find Matching NASA Rows (T_{nasa}):** We convert `acq_date` and `acq_time` in the Excel file into a continuous UTC datetime:

   $$|T_{\text{nasa}} - T_{\text{sat}}| \leq 15 \text{ minutes}$$

   Any detection recorded within 15 minutes of the scan belongs to that satellite frame.

3. **Resample and Project:**

   - Extract $T_{3.9}$ and $T_{10.8}$, compute $\Delta T = T_{3.9} - T_{10.8}$, and stack them into X of shape $(3, 120, 250)$.
   - Convert matching NASA GPS coordinates (lat, lon) into matrix indices (y, x) and set $Y[y, x] = 1.0$:

     $$x = \text{int}\left(\frac{\text{lon} - (-2.0)}{8.5 - (-2.0)} \times 250\right), \quad y = \text{int}\left(\frac{37.5 - \text{lat}}{37.5 - 34.0} \times 120\right)$$

4. **Save Paired Container:** Save X and Y together as a compressed `.npz` file.

3. Preventing Data Leakage (Spatial-Temporal Splitting)

As outlined in Phase 6 of the plan, never use random train/test splitting (such as `train_test_split(shuffle=True)`).

If the satellite scans a large fire at 12:10 UTC and another scan runs at 12:15 UTC, random splitting might place 12:10 in the training set and 12:15 in the test set. The

Split Set	Time Period	Role
Train Set	Summer 2024 & Summer 2025	Learns the convolutional feature representations.
Validation Set	Summer 2026 (June 1 – July 25)	Tunes hyperparameters, learning rate, and early stopping.
Test Set	Summer 2026 (July 26 – September 20)	Evaluates true real-world performance on unseen fire events.

model would score artificially high metrics simply by memorizing the mountain's static geographic features rather than learning the physical signature of combustion.

To guarantee valid real-world generalization, we split strictly by time/year:

The script records this assignment directly into an index file (`dataset_index.csv`) so PyTorch knows which sample belongs to which split.

4. The Complete Processing Script (`build_dataset.py`)

Create a new script named `build_dataset.py` in your project folder.

This script iterates through all `.nat` files in `D:\FINDFIRE_DATA`, matches each to the NASA detections, crops the tensors, and saves the paired training files:

```
from datetime import datetime, timedelta
import glob
import os
import re
import numpy as np
import pandas as pd
from pyresample import create_area_def
from satpy import Scene

#
    ================================================================

# 1. PATHS AND SPATIAL CONFIGURATION
#
    ================================================================

RAW_DATA_DIR = r"D:\FINDFIRE_DATA"
EXCEL_FILE = "nasa data set.xlsx"
OUTPUT_DIR = r"D:\FINDFIRE_DATA\processed"

os.makedirs(OUTPUT_DIR, exist_ok=True)

# Northern Algeria Bounding Box (Phase 3)
WIDTH = 250
HEIGHT = 120
LON_MIN, LON_MAX = -2.0, 8.5
LAT_MIN, LAT_MAX = 34.0, 37.5

algeria_area = create_area_def(
```

```
    area_id="algeria_north",
    projection={"proj": "latlong", "datum": "WGS84"},
    area_extent=[LON_MIN, LAT_MIN, LON_MAX, LAT_MAX],
    shape=(HEIGHT, WIDTH),
    units="degrees",
)

#
    ============================================================================

# 2. LOAD AND INDEX NASA GROUND TRUTH TARGETS
#
    ============================================================================

print(f"Loading NASA targets from: {EXCEL_FILE}...")
df_nasa = pd.read_excel(EXCEL_FILE)

# Construct a single UTC datetime column for every detection
time_str = df_nasa["acq_time"].astype(int).astype(str).str.zfill
    (4)
date_str = df_nasa["acq_date"].astype(str).str[:10]
df_nasa["dt"] = pd.to_datetime(
    date_str + " " + time_str, format="%Y-%m-%d %H%M"
)

# Keep only detections inside Northern Algeria
df_nasa = df_nasa[
    (df_nasa["latitude"] >= LAT_MIN)
    & (df_nasa["latitude"] <= LAT_MAX)
    & (df_nasa["longitude"] >= LON_MIN)
    & (df_nasa["longitude"] <= LON_MAX)
].copy()
print(
    f"Loaded {len(df_nasa):,} valid fire points within Northern
        Algeria"
    " bounds.\n"
)

#
    ============================================================================

# 3. DISCOVER ALL DOWNLOADED .NAT FILES
#
    ============================================================================

nat_files = sorted(glob.glob(os.path.join(RAW_DATA_DIR, "*.nat"))
    )
print(f"Found {len(nat_files)} satellite .nat files to process.")

if not nat_files:
  raise FileNotFoundError(
```

```
        f"No .nat files found in {RAW_DATA_DIR}. Check your folder
            path!"
    )


# Helper: Extract datetime from EUMETSAT filename
def parse_sat_filename(filename):
    basename = os.path.basename(filename)
    # Pattern matches: -YYYYMMDDHHMMSS.
    match = re.search(
        r"-(\d{4})(\d{2})(\d{2})(\d{2})(\d{2})(\d{2})\.", basename
    )
    if match:
        y, m, d, h, mn, s = match.groups()
        return datetime(int(y), int(m), int(d), int(h), int(mn), int(
            s))
    return None


#
    ================================================================

# 4. PROCESSING LOOP (TENSOR EXTRACTION & PAIRING)
#
    ================================================================

dataset_records = []
total_files = len(nat_files)

print("Beginning automated extraction and spatial alignment...\n"
    )

for idx, nat_path in enumerate(nat_files):
    sat_dt = parse_sat_filename(nat_path)
    if sat_dt is None:
        print(f"[{idx+1}/{total_files}] Skipping unparseable file: {
            nat_path}")
        continue

    sample_id = f"sample_{idx:03d}"
    output_npz_path = os.path.join(OUTPUT_DIR, f"{sample_id}.npz")

    # Skip if already processed
    if os.path.exists(output_npz_path):
        print(f"[{idx+1}/{total_files}] {sample_id} already exists.
            Skipping.")
        continue

    # 4A. Query matching NASA fire detections (+/- 15 minutes of
        satellite scan)
    window_start = sat_dt - timedelta(minutes=15)
```

```
    window_end = sat_dt + timedelta(minutes=15)
    matching_fires = df_nasa[
        (df_nasa["dt"] >= window_start) & (df_nasa["dt"] <=
            window_end)
    ]

    # 4B. Build the Ground Truth Binary Mask Y [120, 250]
    mask_y = np.zeros((HEIGHT, WIDTH), dtype=np.float32)
    for _, fire in matching_fires.iterrows():
        lat = fire["latitude"]
        lon = fire["longitude"]

        # Mapping equations
        x = int(((lon - LON_MIN) / (LON_MAX - LON_MIN)) * WIDTH)
        y = int(((LAT_MAX - lat) / (LAT_MAX - LAT_MIN)) * HEIGHT)

        # Clamp within array bounds
        x = min(max(x, 0), WIDTH - 1)
        y = min(max(y, 0), HEIGHT - 1)

        mask_y[y, x] = 1.0

    fire_pixel_count = int(np.sum(mask_y == 1.0))

    # 4C. Load and Resample Satellite Bands via Satpy
    try:
        scn = Scene(filenames=[nat_path], reader="seviri_l1b_native")
        scn.load(["IR_039", "IR_108"])
        local_scn = scn.resample(algeria_area, resampler="nearest")

        t_39 = local_scn["IR_039"].values.astype(np.float32)
        t_108 = local_scn["IR_108"].values.astype(np.float32)

        # Compute physical differential feature
        delta_t = (t_39 - t_108).astype(np.float32)

        # Replace boundary NaNs with 0.0 to prevent gradient
            corruption
        t_39 = np.nan_to_num(t_39, nan=0.0)
        t_108 = np.nan_to_num(t_108, nan=0.0)
        delta_t = np.nan_to_num(delta_t, nan=0.0)

        # Stack into 3-channel input tensor X [3, 120, 250]
        tensor_x = np.stack([t_39, t_108, delta_t], axis=0)

    except Exception as e:
        print(f"[{idx+1}/{total_files}] Failed to process {nat_path}:
            {e}")
        continue

    # 4D. Determine Split (Spatial-Temporal splitting by Year)
```

```
  if sat_dt.year in [2024, 2025]:
    split = "train"
  elif sat_dt.year == 2026 and sat_dt < datetime(2026, 7, 25):
    split = "val"
  else:
    split = "test"

  # 4E. Save Compressed Pair (.npz)
  np.savez_compressed(
      output_npz_path,
      image=tensor_x,  # Shape: [3, 120, 250]
      mask=mask_y,  # Shape: [120, 250]
      timestamp=sat_dt.isoformat(),
      fire_pixels=fire_pixel_count,
  )

  dataset_records.append({
      "sample_id": sample_id,
      "file_path": output_npz_path,
      "timestamp": sat_dt.isoformat(),
      "year": sat_dt.year,
      "fire_pixels": fire_pixel_count,
      "split": split,
  })

  print(
      f"[{idx+1}/{total_files}] Created {sample_id} ({sat_dt.
         strftime('%Y-%m-%d %H:%M')}) "
      f"-> Fires: {fire_pixel_count} px | Split: {split}"
  )

#
   ============================================================================

# 5. SAVE DATASET INDEX CSV
#
   ============================================================================

if dataset_records:
  index_df = pd.DataFrame(dataset_records)
  index_path = os.path.join(OUTPUT_DIR, "dataset_index.csv")
  index_df.to_csv(index_path, index=False)

  print("\n==========================================")
  print("DATASET GENERATION COMPLETE!")
  print(f"Total Samples Generated: {len(index_df)}")
  print("\nSplit Distribution:")
  print(index_df["split"].value_counts())
  print(f"\nIndex saved to: {index_path}")
  print(f"Processed .npz files saved to: {OUTPUT_DIR}")
  print("==========================================")
```

5. Running the Pipeline

Run the script from your terminal:

```
python build_dataset.py
```

- **Execution Time:** Each file takes roughly 1.5 to 2 seconds for Satpy to decode and resample. 360 files will finish processing in approximately 8 to 12 minutes.
- **Output:** You will have a clean directory at `D:\FINDFIRE_DATA\processed` containing all `.npz` files and a `dataset_index.csv` table detailing your train, validation, and test splits.

Once this finishes running, would you like to build the PyTorch Dataset and DataLoader classes to feed these batches directly into your GPU, or inspect a few randomly generated pairs first? **Is `D:\FINDFIRE_DATA\processed` Necessary?**

Short Answer

Yes, it is practically mandatory.

While you could theoretically skip saving those files and process the raw `.nat` files on the fly during training, doing so would cripple your training pipeline.

The reasons come down to three core deep learning and system architecture fundamentals:

1. The GPU Starvation Problem (Training Speed)

When training a neural network in PyTorch, your GPU processes data in milliseconds. The primary bottleneck is feeding the data to the GPU fast enough so it never sits idle.

Reading raw `.nat` on the fly:

- Every time a batch loads, Python would have to open a 100 MB compressed binary file, parse satellite telemetry, apply inverse Planck calibration, and run bilinear geometric resampling using `satpy`. This takes roughly 1.5 to 2 seconds per image.
- **For 360 images:** 360×2 s $= 720$ seconds (12 minutes) per training epoch just to read the data.
- **Over 50 training epochs:** That is 10 hours spent purely decoding satellite headers while your GPU sits at 0% utilization.

Reading preprocessed `.npz` files:

- A `.npz` file contains the already-cropped 120×250 matrix saved as raw float numbers. Loading it from disk takes 0.001 seconds (1 millisecond).
- An entire epoch across all 360 images finishes in less than half a second.

2. RAM and DataLoader Worker Crashes

PyTorch uses multiple parallel background processes (`num_workers` in DataLoader) to prepare batches ahead of time.

- If 4 background workers each attempt to load a raw 1392×3712 full-disk `.nat` file into memory simultaneously, your system RAM will spike by several gigabytes per batch. This frequently triggers Windows memory faults or freezes the Python kernel.
- Loading tiny pre-cut 120×250 arrays uses almost zero RAM, allowing clean, parallel data streaming to your GPU without stability issues.

3. The Disk Space Math (It Costs Almost Nothing)

The raw `.nat` files in `D:\FINDFIRE_DATA` take roughly 36 GB because they hold 12 full planetary channels.

The `processed` directory discards the 9 irrelevant channels and crops out Europe and the ocean. A single sample containing the 3 input channels $[3, 120, 250]$ and the target mask $[120, 250]$ is only around 100 KB:

$$360 \text{ files} \times 100 \text{ KB} \approx 36 \text{ MB}$$

The entire `processed` directory requires less than 50 MB of storage—a negligible cost for eliminating hours of redundant computation during training. Once generated, you can even archive or delete the heavy raw `.nat` files to free up that 36 GB of space